

Java Programing: An Introduction



Faculty of Computer Science and Mathematics, University of Kufa, Najaf, IRAQ.

2nd Class / Computer
Science Dept.

Lecture one:
((Introduction to JAVA))

Dr. Ahmed J. Al-Janaby

Ahmedj.aljanaby@uokufa.edu.iq

A program is written in a particular programming language that uses specific words and symbols to express the problem solution. A programming language defines a set of rules that determine exactly how a programmer can combine the words and symbols of the language into programming statements, which are the instructions that are carried out when the program is executed.

Java is a relatively new programming language compared to others. It was developed in the early 1990s by James Gosling at Sun Microsystems. Java was introduced to the public in 1995 and has gained tremendous popularity since.

Java is used in commercial environments all over the world. It is one of the fastest growing programming technologies of all time. So not only is it a good language in which to learn programming concepts, it is also a practical language that will serve you well in the future.

Java is an **object-oriented** programming language. Objects are the fundamental pieces that make up a program. Other programming languages, such as C++, allow a programmer to use objects but don't **reinforce** تعزيز استخدام، تدعم that approach, which can lead to confusing program designs.

- ***In the Java programming language:***
 - ***A program is made up of one or more classes***
 - ***A class contains one or more methods***
 - ***A method contains program statements***

1.1 Structure of Java Program

```

//*****
// Demonstrates the basic structure of a Java application.
//*****

public class Prog1
{
    //-----
    // Prints a presidential quote.
    //-----
    public static void main (String[] args)
    {
        System.out.println ("A quote by united states:");
        System.out.println ("Whatever you are, be a good one.");
    }
}

```

Output

```

A quote by united states:
Whatever you are, be a good one.

```

```
public class MyProgram
```

Every Java program consists of at least **one class** that you (the programmer) define. The **class** keyword introduces a class declaration and is immediately followed by the **Class** name.

```
public static void main (String[] args)
```

The parentheses **الاقواس** after the identifier **main** indicate that it's a program building block called a **method**. Java class declarations normally contain one or more **methods**.

```
System.out.println ("A quote by united states:");
```

The **System.out** object is known as the standard output object. It allows a Java application to display information in the command window from which it executes.

1.2 Printing a line of text with multiple statements

```

1  // Fig. 2.3: Welcome2.java
2  // Printing a line of text with multiple statements.
3
4  public class Welcome2
5  {
6      // main method begins execution of Java application
7      public static void main( String[] args )
8      {
9          System.out.print( "Welcome to " );
10         System.out.println( "Java Programming!" );
11     } // end method main
12 } // end class Welcome2

```

Welcome to Java Programming!

The backslash (\) is an escape character. which has special meaning to System.

out's print and println methods.

```

1  // Fig. 2.4: Welcome3.java
2  // Printing multiple lines of text with a single statement.
3
4  public class Welcome3
5  {
6      // main method begins execution of Java application
7      public static void main( String[] args )
8      {
9          System.out.println( "Welcome\nto\nJava\nProgramming!" );
10     } // end method main
11 } // end class Welcome3

```

Welcome
to
Java
Programming!

1.3 Addition program that displays the sum of two numbers.

```

1 // Fig. 2.7: Addition.java
2 // Addition program that displays the sum of two numbers.
3 import java.util.Scanner; // program uses class Scanner
4
5 public class Addition
6 {
7     // main method begins execution of Java application
8     public static void main( String[] args )
9     {
10         // create a Scanner to obtain input from the command window
11         Scanner input = new Scanner( System.in );
12
13         int number1; // first number to add
14         int number2; // second number to add
15         int sum; // sum of number1 and number2
16
17         System.out.print( "Enter first integer: " ); // prompt
18         number1 = input.nextInt(); // read first number from user
19
20         System.out.print( "Enter second integer: " ); // prompt
21         number2 = input.nextInt(); // read second number from user
22
23         sum = number1 + number2; // add numbers, then store total in sum
24
25         System.out.printf( "Sum is %d\n", sum ); // display sum
26     } // end method main
27 } // end class Addition

```

```

Enter first integer: 45
Enter second integer: 72
Sum is 117

```

E.x: *System.out.printf("Sum is %d\n", (number1 + number2));*

Scanner input = new Scanner(System.in);

is a variable declaration statement that specifies the name (**input**) and type (**Scanner**) of a variable that's used in this program. A **Scanner** enables a program to read data (e.g., numbers and strings) for use in a program. The data can come from many sources, such as the user at the **keyboard** or a **file** on disk. Before using a Scanner, you must create it and specify the **source of the data**.

1.4 Arithmetic operators:

Java operation	Operator	Algebraic expression	Java expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

1.5 Equality and relational operators:

Standard algebraic equality or relational operator	Java equality or relational operator	Sample Java condition	Meaning of Java condition
<i>Equality operators</i>			
=	==	<code>x == y</code>	x is equal to y
≠	!=	<code>x != y</code>	x is not equal to y
<i>Relational operators</i>			
>	>	<code>x > y</code>	x is greater than y
<	<	<code>x < y</code>	x is less than y
≥	>=	<code>x >= y</code>	x is greater than or equal to y
≤	<=	<code>x <= y</code>	x is less than or equal to y

1.6 Precedence of arithmetic operators:

Operator(s)	Operation(s)	Order of evaluation (precedence)
* / %	Multiplication Division Remainder	Evaluated first. If there are several operators of this type, they're evaluated from left to right.
+ -	Addition Subtraction	Evaluated next. If there are several operators of this type, they're evaluated from left to right.
=	Assignment	Evaluated last.

```

1 // Fig. 2.15: Comparison.java
2 // Compare integers using if statements, relational operators
3 // and equality operators.
4 import java.util.Scanner; // program uses class Scanner
5
6 public class Comparison
7 {
8     // main method begins execution of Java application
9     public static void main( String[] args )
10    {
11        // create Scanner to obtain input from command line
12        Scanner input = new Scanner( System.in );
13
14        int number1; // first number to compare
15        int number2; // second number to compare
16
17        System.out.print( "Enter first integer: " ); // prompt
18        number1 = input.nextInt(); // read first number from user
19
20        System.out.print( "Enter second integer: " ); // prompt
21        number2 = input.nextInt(); // read second number from user
22
23        if ( number1 == number2 )
24            System.out.printf( "%d == %d\n", number1, number2 );
25
26        if ( number1 != number2 )
27            System.out.printf( "%d != %d\n", number1, number2 );
28
29        if ( number1 < number2 )
30            System.out.printf( "%d < %d\n", number1, number2 );
31
32        if ( number1 > number2 )
33            System.out.printf( "%d > %d\n", number1, number2 );
34
35        if ( number1 <= number2 )
36            System.out.printf( "%d <= %d\n", number1, number2 );
37
38        if ( number1 >= number2 )
39            System.out.printf( "%d >= %d\n", number1, number2 );
40    } // end method main
41 } // end class Comparison

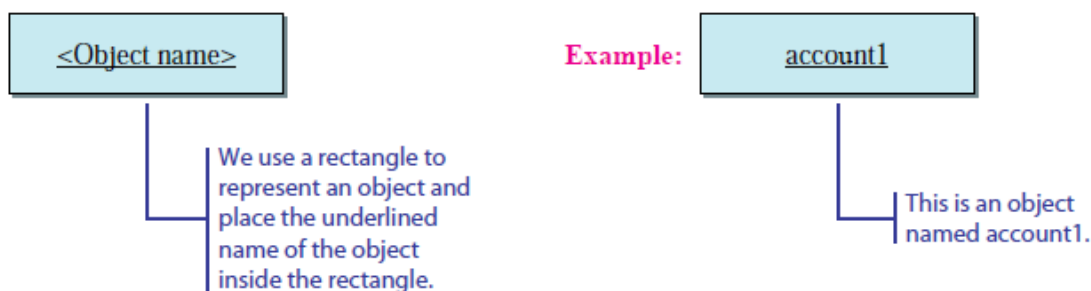
```

1. Introduction to objects

Java is an object-oriented language. As the name implies, an object is a fundamental entity in a Java program. The two most important concepts in object-oriented programming are the **class** and the **object**. A program written in object-oriented style will consist of interacting objects.

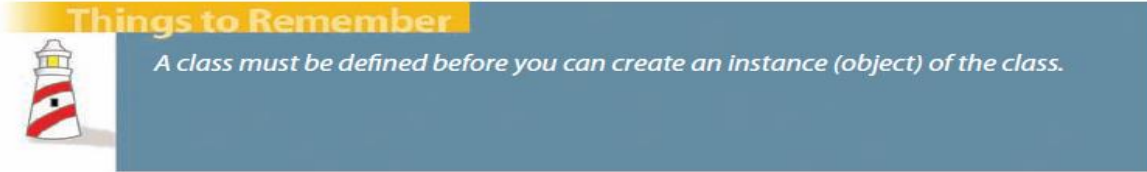
For a program to keep track of student residents of a college dormitory, we may have many *Student*, *Room*, and *Floor* objects. For another program to keep track of customers and inventory for a *bicycle shop*, we may have *Customer*, *Bicycle*, and many other types of objects.

An **object** is comprised of data and operations that manipulate these data. For example, a *Student object* may consist of data such as *name*, *gender*, *birth date*, *home address*, *phone number*, and *age* and operations for assigning and changing these data values.



A graphical representation of an object.

Inside a program we write instructions to create **objects**. For the computer to be able to create an **object**, we must provide a definition called a **class**.



Things To Remember

Message

In writing object-oriented programs we must first define classes, and while the program is running, we use the classes and objects from these classes to accomplish tasks. A task can range from **adding two numbers**, to computing a **payment for a college**, etc. To instruct a class or an object to perform a task, we send a message to it. For example, we send a message deposit to an **Account object** to deposit \$100.

Method

Instance Method

For a class or an object to process the **message** it receives, it must possess a **matching method**, which is a sequence of instructions that a class or an object follows to perform a task. A method defined for a class is called a **class method**. and a **method** defined for an object is an **instance method**

Let's look at an example of an instance method, suppose a **method called walk** is defined for a **Robot object** and instructs the robot to walk a designated distance. With this method

defined, we can send the message walk to a **Robot object**, along with the distance to be walked. A value we pass to an object is called an **argument** of a **message**.